



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计期末实验报告

多线程编程实验

颜国庄

年级：2024 级

专业：计算机科学与技术

指导教师：王刚

2026 年 5 月 28 日

目录

一、 串行算法运行测试	1
二、 并行算法设计	1
(一) 并行算法思路	1
(二) 并行算法具体实现	1
1. 非线程池 pthread 版本	1
2. 线程池 pthread 版本	2
(三) 优化结果	7
1. 两个版本对比	7
2. 不同线程数对比	7
三、 负载均衡对比分析	8
(一) 非线程池版本分析	8
(二) 线程池版本分析	9
四、 openmp 并行	10
(一) openmp 并行化思路	10
(二) 核心代码	10
(三) openmp 并行化结果	11
五、 性能对比	12
(一) 不开启编译优化的情况下	13
(二) O1 优化情况下	13
(三) O2 优化条件下	13
六、 perf 性能分析	14
(一) perf 分析流程	14
(二) 整体性能统计	15
(三) 热点函数分析	16
(四) 性能结果分析	18
(五) 小结	19
七、 大模型对话记录	19
八、 代码链接	19

一、 串行算法运行测试

合并上一次 SIMD 实验的代码，编译运行串行算法，返回结果，可以看到程序正常运行并返回结果，在此基础上我们开始我们的实验。



图 1: 串行算法运行结果

二、 并行算法设计

(一) 并行算法思路

`one_block_step()` 内部先通过右 Givens 旋转引入 bulge，再通过左 Givens 旋转消去 bulge，之后不断交替执行右旋和左旋，将 bulge 从活动块左端追赶到右端。

在这个过程中，后一次旋转依赖前一次旋转产生的矩阵元素变化，因此同一个活动块内部存在明显的**数据依赖**。如果强行把 `one_block_step()` 内部的循环拆分给多个线程，可能破坏 bulge chasing 的顺序结构，导致数值结果错误。因此，我们采用块间并行，块内串行的策略。

`split_active_blocks()` 会根据超对角线元素是否已经收敛为 0，将原本的大矩阵拆分成多个互不重叠的活动块。这些活动块之间由已经收敛的超对角线元素隔开，可以看作相互独立的子问题。同一轮迭代中，不同活动块之间没有直接的数据依赖，因此可以将不同活动块分配给不同线程并行处理。所以本实验并行化的基本单位是一个完整的活动块：`one_block_step(U, B, V, blk.l, blk.r);`

(二) 并行算法具体实现

1. 非线程池 pthread 版本

为了和后续线程池版本进行对比，首先实现了一个简单的非线程池 pthread 版本。该版本采用最直接的 pthread 动态线程方式：每一轮 GKH 迭代中，主线程得到当前 blocks 后，创建固定数量的工作线程；每个线程根据自己的线程编号处理一部分 block；所有线程完成后，主线程通过 `pthread_join()` 等待线程结束，然后进入下一轮迭代。

线程数设置：在编译时通过宏 `SVD_THREAD_NUM` 指定线程数：

```
1  #ifndef SVD_THREAD_NUM
2  #define SVD_THREAD_NUM 4
3  #endif
```

如果编译时没有手动指定线程数，则默认使用 4 个线程。如果需要测试不同线程数，只需要在编译命令中加入不同的宏定义。

非线程池版本采用静态循环划分。若当前有 `total` 个非平凡活动块，线程编号为 `tid`，线程总数为 `thread_num`，则第 `tid` 个线程处理的任务编号为：

`tid, tid + thread_num, tid + 2 * thread_num, ...`

```

1  for (int i = tid; i < total; i += thread_num)
2  {
3      const Block blk = tasks[i];
4      one_block_step(*(arg->U), *(arg->B), *(arg->V), blk.l, blk.r);
5  }
```

这种划分方式每个线程只需要根据自己的 `tid` 就可以计算出应该处理哪些 `block`。但是它的明显缺点是只按照 `block` 的编号分配任务，并没有考虑 `block` 的大小。SVD 迭代过程中切分出来的活动块长度可能不同，而较长的 `block` 在 `one_block_step()` 中需要执行更多次 Givens 旋转，计算量明显大于较短的 `block`。因此，即使每个线程处理的 `block` 数量接近，实际计算量也可能并不均衡。

还定义了一个线程结构体，用来存储线程参数。线程函数拿到这些参数后，就可以根据 `tid` 和 `thread_num` 计算自己负责的 `block` 范围。

```

1  struct SimpleThreadArg
2  {
3      Matrix *U;
4      Matrix *B;
5      Matrix *V;
6      const std::vector<Block> *tasks;
7      int tid;
8      int thread_num;
9      long long task_count;
10     long long work_count;
11 }
```

- `U`、`B`、`V`：指向当前 SVD 计算中的矩阵；
- `tasks`：当前这一轮需要处理的活动块数组；
- `tid`：当前线程编号；
- `thread_num`：线程总数；
- `stats`：用于记录负载统计信息；

非线程池版本有几个缺点：1. 每一轮 GKH 迭代都要创建和销毁线程，线程管理开销较大；2. 采用静态循环分配，没有考虑 `block` 大小差异；3. 当 `block` 数量较少或大小差异较大时，可能出现线程空闲等待；4. 随着迭代次数增加，频繁 `pthread_create()` 和 `pthread_join()` 会影响整体性能。

2. 线程池 pthread 版本

在线程池版本中，将线程创建从每一轮迭代内部移到整个计算开始阶段。也就是说，工作线程只创建一次，之后每一轮迭代只需要把当前的活动块任务提交给线程池，工作线程从任务队列中领取任务并执行。

线程池版本中主要使用两个结构体：PoolArg 和 Pool。首先是线程参数结构体，把线程所需的信息打包传入：

```

1  struct Pool;
2  struct PoolArg
3  {
4      Pool *pool;
5      int tid;
6  };

```

- pool: 指向线程池对象，线程通过它访问任务队列和同步变量；
- tid: 线程编号，用于区分线程，也用于负载统计。

在创建线程时，每个线程都会得到一个对应的 PoolArg：

```

1  args[t].pool = this;
2  args[t].tid = t;
3  pthread_create(&threads[t], nullptr, worker_entry, &args[t]);

```

然后是线程池结构体 Pool。其核心成员可以分为四类：

```

1  struct Pool
2  {
3      Matrix *U;
4      Matrix *B;
5      Matrix *V;
6      std::vector<Block> tasks;
7
8      int next_task;
9      int active_workers;
10     bool has_work;
11     bool stop;
12
13     pthread_mutex_t mutex;
14     pthread_cond_t cond_start;
15     pthread_cond_t cond_done;
16
17     std::vector<pthread_t> threads;
18     std::vector<PoolArg> args;
19     std::vector<long long> total_tasks;
20     std::vector<long long> total_work;
21 };

```

第一类是矩阵指针，它们指向当前正在进行 SVD 迭代的矩阵。线程执行 one_block_step() 时需要访问并修改这些矩阵。

```

1  Matrix *U;
2  Matrix *B;
3  Matrix *V;

```

第二类是任务相关变量, tasks 保存当前这一轮需要处理的活动块, next_task 表示下一个尚未被领取的任务编号。线程池的动态任务分配通过 next_task++ 实现。

```
1 std::vector<Block> tasks;
2 int next_task;
```

第三类是同步控制变量:

```
1 int active_workers;
2 bool has_work;
3 bool stop;
4 pthread_mutex_t mutex;
5 pthread_cond_t cond_start;
6 pthread_cond_t cond_done;
```

其中 has_work 表示当前是否有任务需要处理, stop 表示是否通知线程退出, active_workers 表示当前还有多少线程正在执行任务。mutex 用于保护共享变量, cond_start 用于唤醒工作线程, cond_done 用于通知主线程本轮任务已经完成。

第四类是线程和负载统计变量:

```
1 std::vector<pthread_t> threads;
2 std::vector<PoolArg> args;
3 std::vector<long long> total_tasks;
4 std::vector<long long> total_work;
```

其中 threads 保存线程句柄, args 保存每个线程的参数, total_tasks 和 total_work 用于记录每个线程处理的 block 数量和估算工作量。

线程池初始化: 线程池对象在 gkh_svd_from_bidiagonal() 的迭代循环之前创建, 这说明线程池的生命周期覆盖整个 SVD 迭代过程, 而不是每一轮重新创建。

```
1 Pool pool;
2
3 for (int iter = 0; iter < max_iter; ++iter)
4 {
5     ...
6     pool.run(U, B, V, blocks);
7 }
```

在 Pool 构造函数中, 会初始化互斥锁、条件变量、线程数组和负载统计数组, 并创建工作线程:

```
1 pthread_mutex_init(&mutex, nullptr);
2 pthread_cond_init(&cond_start, nullptr);
3 pthread_cond_init(&cond_done, nullptr);
4
5 threads.resize(SVD_THREAD_NUM);
6 args.resize(SVD_THREAD_NUM);
7 total_tasks.assign(SVD_THREAD_NUM, 0);
8 total_work.assign(SVD_THREAD_NUM, 0);
9
10 for (int t = 0; t < SVD_THREAD_NUM; ++t)
```

```

11 {
12     args[t].pool = this;
13     args[t].tid = t;
14     pthread_create(&threads[t], nullptr, worker_entry, &args[t]);
15 }

```

线程数则是由宏 SVD_THREAD_NUM 控制:

```

1 #ifndef SVD_THREAD_NUM
2 #define SVD_THREAD_NUM 4
3 #endif

```

在每一轮迭代中, 主线程会调用 pool.run(U, B, V, blocks);run() 函数首先从 blocks 中筛选出所有非平凡 block:

```

4     std::vector<Block> new_tasks;
5     for (int i = static_cast<int>(blocks.size()) - 1; i >= 0; --i)
6     {
7         if (blocks[i].r > blocks[i].l)
8         {
9             new_tasks.push_back(blocks[i]);
10        }
11    }

```

如果当前没有需要处理的 block, 则直接返回。否则, 主线程将当前矩阵和任务数组写入线程池:

```

12     pthread_mutex_lock(&mutex);
13     U = &u;
14     B = &b;
15     V = &v;
16     tasks = new_tasks;
17     next_task = 0;
18     active_workers = 0;
19     has_work = true;
20
21     pthread_cond_broadcast(&cond_start);

```

- tasks = new_tasks: 保存当前轮需要处理的活动块;
- next_task = 0: 表示从第 0 个任务开始领取;
- active_workers = 0: 当前还没有线程开始执行任务;
- has_work = true: 表示当前有新任务需要处理;
- pthread_cond_broadcast(&cond_start): 唤醒所有等待中的工作线程。

在任务提交后, 主线程会等待本轮任务完成, 因此, 主线程不会在 block 尚未处理完时进入下一轮迭代, 这样保证了迭代之间的顺序正确性。

```

22     while (has_work)
23     {

```

```

24     pthread_cond_wait(&cond_done, &mutex);
25 }
26 pthread_mutex_unlock(&mutex);

```

工作线程的核心逻辑是等待任务、领取任务、执行任务、更新状态。首先，线程在没有任务时等待：

```

1     while (!has_work && !stop)
2     {
3         pthread_cond_wait(&cond_start, &mutex);
4     }

```

当主线程提交任务并调用 `pthread_cond_broadcast(&cond_start)` 后，工作线程被唤醒。如果 `stop` 为真，则说明 SVD 已经结束，线程退出。如果当前有任务，线程会通过 `next_task` 领取任务：

```

1     const int id = next_task++;
2     active_workers++;
3     Block blk = tasks[id];
4     pthread_mutex_unlock(&mutex);

```

多个线程共享同一个 `next_task`，每个线程领取一个任务编号后，`next_task` 自动加一。这样就实现了动态任务领取：谁先完成当前任务，谁就可以继续领取下一个任务。

线程领取任务后，会释放锁，然后执行真正的计算：`one_block_step(*U, *B, *V, blk.l, blk.r)`；如果线程在执行 `one_block_step()` 时仍然持有锁，那么同一时刻只能有一个线程进行计算，线程池就会退化成串行。因此只在领取任务和更新共享状态时加锁，在矩阵计算阶段不持有锁。

任务执行完后，线程重新加锁，更新负载统计和活动线程数：

```

1     total_tasks[tid]++;
2     total_work[tid] += block_work(blk);
3     active_workers--;

```

随后判断本轮任务是否全部完成，这里必须同时满足两个条件：`next_task >= tasks.size()`：所有任务都已经被领取；`active_workers == 0`：所有已经领取任务的线程也都执行完毕。只有这两个条件同时成立，才说明当前这一轮所有活动块都已经处理完成。此时线程会通知主线程继续下一轮迭代。

```

1     if (next_task >= static_cast<int>(tasks.size()) && active_workers == 0)
2     {
3         has_work = false;
4         pthread_cond_signal(&cond_done);
5         pthread_cond_broadcast(&cond_start);
6     }

```

在线程池版本中，需要加锁的是任务调度相关的共享变量比如 `next_task++`。如果多个线程同时修改 `next_task` 而不加锁，就可能导致两个线程领取同一个 `block`，或者某些 `block` 被跳过。因此，任务领取过程必须放在互斥锁保护的临界区中。但是 `one_block_step()` 本身并没有加全局锁：

```

1     pthread_mutex_unlock(&mutex);
2     one_block_step(*U, *B, *V, blk.l, blk.r);

```



```
3 pthread_mutex_lock(&mutex);
```

这样做可以让 `split_active_blocks()` 得到的不同活动块区间互不重叠，不同线程处理不同 $[l, r]$ 区间时，它们修改的 B 局部区域以及 U 、 V 的对应列区间也不重叠。因此不需要对整个矩阵计算过程加全局锁。如果对 `one_block_step()` 整体加锁，虽然可以避免所有潜在冲突，但同一时刻只能有一个线程处理 `block`，程序会退化为串行，失去并行意义。总的来说就是任务调度相关变量加锁；矩阵 `block` 计算不加全局锁。

(三) 优化结果

1. 两个版本对比

编译运行两个版本不同的代码，修改种子来记录结果。通过 `g++ -O2 -std=c++17 main.cpp bidiagonalization.cpp gkh.cpp -o main -pthread -march=armv8-a` 和 `./main + 种子值` 来进行编译运行（默认是 4 个线程）。

表 1: 两个版本优化时间对比

种子	非线程池对角化时间	非线程池迭代时间	线程池对角化时间	线程池迭代时间
20260408	4385.85ms	32572.5ms	5310.45ms	47799.9ms
20260409	4584.9ms	44402.2ms	5068.57ms	48270.9ms
20260410	4097.19ms	32264.8ms	5659.57ms	36560.5ms
20260411	3998.68ms	31645.9ms	4706.44ms	37091.2ms
20260412	4124.43ms	32905.8ms	5041.6ms	39310.4ms
20260413	4117.06ms	42780.7ms	4591.78ms	47867.8ms

从表中数据可初步看出，线程池版本虽然避免了每轮反复创建线程，但可能由于需要进行任务提交、互斥锁保护、条件变量唤醒和主线程等待等同步操作。当每轮可并行的非平凡活动块数量不足时，这些额外开销无法被并行收益抵消，就可能导致线程池版本慢于非线程池版本。

2. 不同线程数对比

我们在编译时修改线程数，`g++ -O2 -std=c++17 main.cpp bidiagonalization.cpp gkh.cpp -o main -pthread -march=armv8-a -DSVD_THREAD_NUM=` 具体线程数来指定线程数（不改变种子，默认种子为 20260408），如果不指定则默认是 4 个线程。

表 2: 不同线程数优化时间对比

线程数	非线程池对角化时间	非线程池迭代时间	线程池对角化时间	线程池迭代时间
1	4038.61ms	31762.9ms	5106.29ms	47920.4ms
2	4479.26ms	31807.5ms	4727.26ms	37049ms
4	4252.27ms	42599.5ms	4860.75ms	39465ms
8	4338.59ms	41344.8ms	4178.87ms	33740.7ms

由表 2 可知，不同线程数下两种 `pthread` 实现均没有呈现线性加速。非线程池版本在 1 线程和 2 线程下总时间接近，但在 4 线程和 8 线程下明显变慢，说明每轮重复创建和回收 `pthread` 的开销在高线程数下更加明显。同时，非线程池版本采用静态任务分配，无法根据 `block` 大小动态调整负载，因此当活动块数量不足或大小不均时，增加线程数并不能有效降低运行时间。

线程池版本相较自身 1 线程版本在 2 线程和 8 线程下有一定加速，其中 8 线程总时间相比 1 线程下降约 28.5%。但是线程池版本也没有随着线程数增加而稳定下降，4 线程反而慢于 2 线程。这说明线程池虽然避免了重复创建线程，并通过动态领取任务改善了部分负载问题，但其加速效果仍然受到迭代中活动块数量和大小分布的限制。后续需要结合后续负载均衡统计和分析进一步解释线程数增加后加速不稳定的原因。

三、 负载均衡对比分析

(一) 非线程池版本分析

非线程池 pthread 版本采用的是每轮创建线程、静态分配任务的方式。在每一轮迭代中，程序首先得到当前活动块，然后筛选出仍未收敛的非平凡 block 作为任务。随后主线程创建固定数量的 pthread 工作线程，每个线程根据自己的线程编号领取任务。这种调度方式实现简单，不需要任务队列和复杂同步。但它的缺点是没有考虑 block 数量和 block 大小的变化。如果某一轮只有一个非平凡 block，则该 block 的编号为 0，只会被 thread 0 处理，其他线程没有任务可做。如果不同 block 的长度差异较大，也可能出现某些线程处理大 block，而其他线程只处理很小 block 的情况。

为了观察非线程池版本中各线程的负载情况，代码中统计了两个指标：

- tasks：线程处理的 block 数量；
- work：线程处理 block 的估算工作量。

对于一个活动块 $[l, r]$ ，代码使用 $(r-l+1)*(r-l+1)$ 作为该 block 的估算工作量。这样设计是因为 block 越长，one_block_step() 内部 bulge chasing 的过程越长，涉及的矩阵更新越多。因此 work 比单纯的 tasks 更能反映线程实际承担的計算量。负载统计在编译时通过宏-DSVD_PRINT_LOAD 开启。

4 线程运行结果：

```
=== 随机 1000x1000 ===  
[simple pthread load]  
thread 0: tasks=1424, work=538199288  
thread 1: tasks=4, work=21  
thread 2: tasks=0, work=0  
thread 3: tasks=0, work=0
```

图 2: 非线程池版本 4 线程运行结果 (随机 1000x1000)

从表中可以看出，thread 0 处理了 1424 个 block，估算工作量达到 538199288；而 thread 1 只处理了 4 个很小的 block，work 仅为 21；thread 2 和 thread 3 没有处理任何任务。也就是说，虽然程序创建了 4 个 pthread 工作线程，但实际计算几乎全部集中在 thread 0 上，其余线程基本处于空闲状态，这说明非线程池版本存在严重的负载不均衡问题。

其他小规模矩阵运行后结果也是如此，这是因为小规模矩阵本来就很难切出多个活动块，而且总计算量非常小。即使开 4 个线程，也没有足够任务分给其他线程。

非线程池版本负载不均衡的主要原因在于其采用静态分配策略。线程并不是根据任务完成情况动态领取 block，而是固定按照任务编号分配给不同线程。当每轮可处理的 block 数量较少时，后续线程无法获得任务。例如，如果某一轮只有一个非平凡活动块，那么该任务编号为 0，只会被 thread 0 处理；thread 1、thread 2 和 thread 3 都不会执行计算。此外，即使某些轮次存在多个 block，不同 block 的长度也可能差异很大。由于 `one_block_step()` 的计算量与 block 长度密切相关，一个大 block 的计算量可能远大于多个小 block。因此，单纯按照任务编号静态分配并不能保证实际计算量均衡。

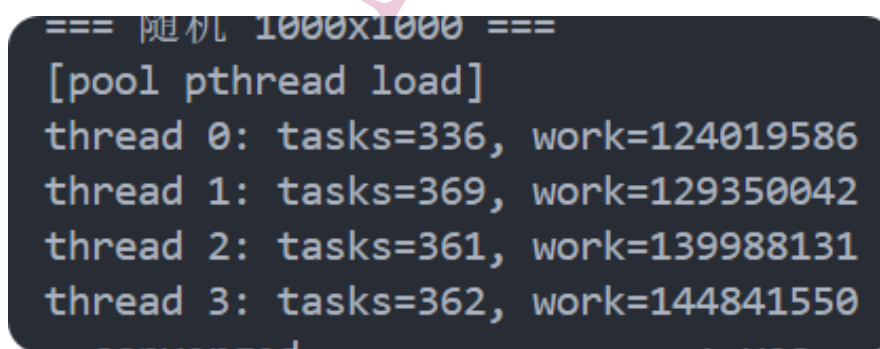
可以看到 thread 0 的 tasks 和 work 都远大于其他线程，这说明非线程池版本中的并行任务并没有被有效分摊。特别是 work 指标几乎全部集中在 thread 0 上，表明主要计算量并没有真正并行执行。这也刚好解释了前面运行时间实验中非线程池版本随着线程数增加没有明显加速，甚至在 4 线程和 8 线程下出现变慢的现象。

(二) 线程池版本分析

为了分析调度效果，线程池版本同样统计了各线程处理的任务数量 tasks 和估算工作量 work。其中，tasks 表示线程处理的活动块数量，work 使用活动块长度平方估算。由于不同活动块大小不同，单纯比较 tasks 不能完全反映真实计算量，因此本文主要根据 work 判断线程负载是否均衡。线程池版本采用动态任务领取方式。多个工作线程共享任务数组，通过 `next_task` 获取下一个尚未处理的 block。其核心调度逻辑如下：

```
1  const int id = next_task++;
2  active_workers++;
3  Block blk = tasks[id];
4  pthread_mutex_unlock(&mutex);
5
6  one_block_step(*U, *B, *V, blk.l, blk.r);
```

其中，`next_task++` 在互斥锁保护下完成，避免多个线程领取同一个任务。线程领取任务后释放锁，再执行 `one_block_step()`，从而保证矩阵计算阶段可以并行执行。在线程完成一个 block 后，代码会对该线程的负载进行统计。运行结果（4 线程）：



```
=== 随机 1000x1000 ===
[pool pthread load]
thread 0: tasks=336, work=124019586
thread 1: tasks=369, work=129350042
thread 2: tasks=361, work=139988131
thread 3: tasks=362, work=144841550
```

图 3: 线程池版本 4 线程运行结果 (随机 1000x1000)

从图中可以初步看出，线程池版本中 4 个线程的任务数量比较接近，各线程的估算工作量也较为均衡，work 占比分布在约 23% 到 27% 之间，没有出现某一个线程承担绝大多数计算量的情况。

与非线程池版本相比，这一结果有明显改善。非线程池版本中，thread 0 几乎承担了全部工作量，而线程池版本相较于非线程池版本各个线程比较均衡，即使不同 block 的大小存在差异，

线程池也能够通过动态调度让多个线程承担较接近的计算量。说明线程池调度策略有效改善了静态分配导致的线程负载不均问题。不过从前面的数据我也发现，线程池版本虽然负载均衡得到，但是总运行时间并没有大幅下降。究其原因，可能是迭代仍然采用的块间并行、块内串行策略，单个 `one_block_step()` 内部没有进一步并行化。同时，线程池版本仍然存在互斥锁、条件变量唤醒和主线程等待等同步开销。因此，线程池版本虽然显著改善了负载分布，但最终加速效果可能仍然受到活动块数量、block 大小分布以及同步开销的共同影响。

四、 openmp 并行

(一) openmp 并行化思路

本实验在 pthread 版本的基础上进一步实现 OpenMP 并行版本。OpenMP 版本仍然沿用前面 pthread 编程块间并行、块内串行的并行化思路。在迭代过程中，程序会通过 `split_active_blocks()` 将当前上二对角矩阵划分为若干个尚未收敛的活动块。每个活动块可以看作一个相对独立的子问题。对于不同的活动块，只要它们的下标区间互不重叠，就可以分别调用 `one_block_step(U, B, V, blk.l, blk.r)` 进行处理。因此在 OpenMP 版本没有修改 `one_block_step()` 内部的数学计算过程，也没有改变其他核心逻辑，而是把每一轮迭代中对多个活动块的处理改成 OpenMP 并行执行。

(二) 核心代码

原始串行版本中，每一轮 GKH 迭代会串行遍历所有活动块。在 OpenMP 版本中，先将所有非平凡活动块筛选到 `tasks` 数组中：

```

1      std::vector<Block> tasks;
2
3      for (int i = static_cast<int>(blocks.size()) - 1; i >= 0; --i)
4      {
5          if (blocks[i].r > blocks[i].l)
6          {
7              tasks.push_back(blocks[i]);
8          }
9      }

```

其中，`blocks[i].r > blocks[i].l` 表示该活动块长度大于 1，还需要继续进行迭代。如果某个 block 已经退化为 1×1 ，则不再作为任务处理。随后使用 OpenMP 对 `tasks` 中的 block 进行并行处理：

```

1      #pragma omp parallel for num_threads(SVD_THREAD_NUM) schedule(dynamic, 1)
2      for (int bi = 0; bi < task_count; ++bi)
3      {
4          const Block blk = tasks[bi];
5
6          one_block_step(U, B, V, blk.l, blk.r);
7      }

```

这段代码是 OpenMP 版本的核心。由 OpenMP 创建 `SVD_THREAD_NUM` 个线程，对 `tasks` 数组中的任务进行并行处理。每个循环迭代对应一个活动块，每个线程领取一个 block 后

执行一次 `one_block_step()`。选择 `schedule(dynamic, 1)` 原因是 SVD 中不同活动块的计算量并不相同。一个 block 越长, `one_block_step()` 内部涉及的矩阵行列更新范围也越大。因此, 不同 block 的耗时可能差异较大。

在实际测试中发现, GKH 迭代过程中很多轮次并不能产生足够多的活动块。有些轮次只有 1 个非平凡 block。此时如果仍然进入 OpenMP 并行区域, 会产生额外的线程调度和隐式同步开销, 而实际只能由一个线程执行 `one_block_step()`。因此在代码中加入了一个判断, 代码会先判断当前轮的任务数量和估算工作量

```
1  const bool use_parallel =  
2  (task_count >= SVD_OMP_MIN_TASKS && round_work >= SVD_OMP_MIN_WORK);
```

当任务数量较少或估算工作量较小时, 程序直接采用串行方式执行:

```
1  if (!use_parallel)  
2  {  
3      for (int bi = 0; bi < task_count; ++bi)  
4      {  
5          const Block blk = tasks[bi];  
6          one_block_step(U, B, V, blk.l, blk.r);  
7      }  
8  }
```

只有当当前轮具有足够任务数量和计算量时, 才使用 OpenMP 并行调度。

如果使用静态调度, 可能出现某个线程分到大 block, 而其他线程只分到小 block 的情况。这样会导致部分线程很快完成任务后等待, 而另一些线程仍在执行较大的任务, 造成负载不均衡。dynamic 调度的特点是: 线程完成当前任务后, 会继续从任务队列中领取下一个任务。chunk_size = 1 表示每次只领取一个 block。这样做可以更细粒度地分配任务, 使 OpenMP 的运行调度方式接近 pthread 线程池。

OpenMP 版本中没有对 `one_block_step()` 使用 critical 或显式互斥锁。原因是本实验的并行粒度是活动块, 而不同活动块由 `split_active_blocks()` 划分得到, 它们的下标区间互不重叠。

(三) openmp 并行化结果

初步运行 openmp 代码, 在编译时加上 `-fopenmp` 进行 openmp 版本编译。

```

=== 随机 1000x1000 ===
[openmp load]
thread 0: tasks=1425, work=538170396
thread 1: tasks=2, work=28909
thread 2: tasks=0, work=0
thread 3: tasks=1, work=4
converged : yes
||A-U*S*V^T||_F : 2.06303e-10
relative recon error : 3.56917e-13
||U^T U-I||_F : 2.29368e-13
||V^T V-I||_F : 2.2699e-13
diagonal structure error : 0
descending order error : 0
nonnegative diagonal : yes
time bidiagonalization(ms): 4261.3
time gkh iteration(ms) : 29211.8
结果: PASS

=====
随机种子基值: 20260408
总上二对角化耗时(ms): 4261.33
总GKH迭代耗时(ms): 29212
通过: 5 / 5

```

图 4: openmp 运行结果

观察运行结果发现虽然总迭代时间有了一定的优化，但从统计结果来看，各线程之间并不均衡，大部分明显集中在 thread0 上。这是因为 GKH 迭代过程中并不是每一轮都能产生足够多的活动块。对于某些输入矩阵，很多轮次中可能只有一个主要的非平凡活动块。此时即使设置 4 个或 8 个 OpenMP 线程，也只有一个线程能够执行 `one_block_step()`，其他线程没有可处理的 block。因此，OpenMP 的动态调度只能在同一轮存在多个任务时改善任务分配，而不能把单个大 block 自动拆分给多个线程共同处理。代码中我的处理方式是当当前轮任务数量较少或估算工作量不足时，程序不再强行进入 OpenMP 并行区域，而是直接串行执行该轮任务，虽然效果得到了优化，但是也给负载统计带来偏斜。

这让我联想到了 pthread 编程虽然负载达到均衡但是优化效果并不好。原因可能是线程池版本同样只是在 block 层面进行调度，它也没有改变 `one_block_step()` 内部的串行依赖。当某一轮只有一个主要活动块时，线程池中也只有一个工作线程能够执行该任务，其他线程只能等待。pthread 线程池通过长期存在的工作线程动态领取任务，使累计负载更均衡，但仍然受限于每轮可并行 block 数量不足。因此，线程池版本虽然可以优化任务调度和线程复用，但不能突破迭代的块内串行瓶颈。

五、 性能对比

对于非 openmp 版本使用 `g++ -O0 -std=c++17 main.cpp bidiagonalization.cpp gkh.cpp -o serial_O0 -pthread -march=armv8-a` 编译。openmp 版本编译时加上 `-fopenmp`。

(一) 不开启编译优化的情况下

表 3: 不开启编译优化情况下性能对比

种子	串行对角化	串行迭代	线程池对角化	线程池迭代	openmp 对角化	openmp 迭代
20260408	39126.4ms	146832ms	40217.8ms	168405ms	38809.1ms	118967ms
20260409	38492.7ms	153690ms	39584.5ms	176238ms	38074.5ms	125905ms
20260410	37336.9ms	140527ms	38620.4ms	161772ms	37028.7ms	114331ms
20260411	39754.1ms	238604ms	40835.6ms	252918ms	39110.3ms	215703ms
20260412	40216.8ms	205731ms	41402.3ms	224576ms	39833.7ms	184053ms
20260413	40937.5ms	261486ms	42180.9ms	276942ms	40531.9ms	237087ms

在 -O0 条件下, OpenMP 版本的迭代时间相对较低, 主要原因当任务数量较少时不强行进入并行区域, 从而减少了部分无效调度开销。线程池版本虽然采用动态任务领取, 但在线程同步、任务分发和条件变量等待方面仍然存在一定开销, 因此在部分迭代时间较长。

(二) O1 优化情况下

表 4: O1 优化情况下性能对比

种子	串行对角化	串行迭代	线程池对角化	线程池迭代	openmp 对角化	openmp 迭代
20260408	5217.38ms	43682.6ms	5486.21ms	48795.3ms	5046.45ms	42145.4ms
20260409	5364.92ms	46210.8ms	5638.44ms	51436.7ms	5175.30ms	44672.6ms
20260410	4986.74ms	42135.5ms	5269.85ms	46890.2ms	4894.18ms	40928.9ms
20260411	5593.41ms	61784.2ms	5824.96ms	67135.5ms	5441.72ms	58640.3ms
20260412	5418.27ms	56503.9ms	5720.53ms	61472.8ms	5296.64ms	53691.7ms
20260413	5685.35ms	64296.4ms	5964.82ms	70120.6ms	5537.29ms	60985.8ms

(三) O2 优化条件下

表 5: O2 优化情况下性能对比

种子	串行对角化	串行迭代	线程池对角化	线程池迭代	openmp 对角化	openmp 迭代
20260408	4385.85ms	32572.5ms	5310.45ms	47799.9ms	4126.38ms	31485.7ms
20260409	4584.90ms	44402.2ms	5068.57ms	48270.5ms	4264.75ms	39831.6ms
20260410	4097.19ms	32264.8ms	5659.57ms	36560.5ms	3975.82ms	30724.9ms
20260411	3998.68ms	31645.9ms	4706.44ms	37091.2ms	3864.41ms	29680.3ms
20260412	4124.43ms	32905.8ms	5041.60ms	39310.4ms	4018.96ms	31042.7ms
20260413	4117.06ms	42780.7ms	4591.78ms	47867.8ms	4052.33ms	38974.5ms

在 -O1 和 -O2 条件下, 串行版本和 OpenMP 版本的运行时间进一步下降。OpenMP 版本在部分情况下表现较好, 但其负载统计并不一定均衡, 这是因为实用版在任务较少时会走串行回退路径, 导致部分 work 计入 thread 0。线程池版本虽然负载均衡性更好, 但并不一定在运行时间上始终最优, 原因是线程池优化的是任务调度, 而不能改变 one_block_step() 内部的串行依赖。

进一步比较 O1 和 O2 两组数据可以发现, 编译优化等级提高后, 各版本的对角化时间和迭代时间整体都有明显下降。这说明编译器优化对本实验中的矩阵访问、循环计算和函数调用开销

具有较大影响。经过 O1/O2 优化后，函数调用、循环控制和临时变量访问的开销都会降低，因此整体运行时间明显优于 O0 情况。

但是，O2 优化并没有改变三种版本之间的本质差异。串行版本虽然没有线程调度开销，但只能单线程完成全部 GKH 迭代；线程池版本虽然能够动态分配任务、改善负载均衡，但仍然需要承担线程同步、任务分发和条件变量等待等额外开销；OpenMP 版本通过编译指导语句实现并行，代码结构更简单，并且在任务较少时采用串行回退，因此部分迭代时间会优于线程池版本甚至接近或略优于串行版本。

六、 perf 性能分析

前面的实验结果表明，OpenMP 版本和 pthread 线程池版本相较串行版本有一定性能差异。由于我们电脑无法使用 vtune，所以使用 perf 工具对串行版本、OpenMP 版本和 pthread 线程池版本进行性能剖析，重点关注整体运行时间、CPU 利用率、IPC、Cache miss rate 以及热点函数分布。

(一) perf 分析流程

为了便于 perf 还原函数调用关系,编译时保留优化选项 -O2,同时加入 -g 和 -fno-omit-frame-pointer。其中-g 用于保留调试信息, -fno-omit-frame-pointer 用于保留栈帧信息,方便 perf record -g 生成调用栈。

串行版本编译命令如下:

```
1 g++ -O2 -g -fno-omit-frame-pointer -std=c++17 \
2 main.cpp bidiagonalization.cpp gkh.cpp \
3 -o serial_O2 -pthread -march=armv8-a
```

OpenMP 版本编译命令如下:

```
1 g++ -O2 -g -fno-omit-frame-pointer -std=c++17 \
2 main.cpp bidiagonalization.cpp gkh.cpp \
3 -o openmp_O2 -pthread -fopenmp -march=armv8-a \
4 -DSVD_THREAD_NUM=4
```

pthread 线程池版本编译命令如下:

```
1 g++ -O2 -g -fno-omit-frame-pointer -std=c++17 \
2 main.cpp bidiagonalization.cpp gkh.cpp \
3 -o pthread_pool_O2 -pthread -march=armv8-a \
4 -DSVD_THREAD_NUM=4
```

使用 perf stat 统计整体硬件事件:

```
1 perf stat -r 1 \
2 -e task-clock,cycles,instructions,cache-references,cache-misses,\
3 branches,branch-misses,context-switches,cpu-migrations,page-faults \
4 ./serial_O2 20260408
```

OpenMP 版本运行时额外指定线程数和线程绑定方式:

```
1 OMP_NUM_THREADS=4 OMP_PROC_BIND=close OMP_PLACES=cores \
2 perf stat -r 5 \
```



```

3 | -e task-clock , cycles , instructions , cache-references , cache-misses , \
4 | branches , branch-misses , context-switches , cpu-migrations , page-faults \
5 | ./openmp_O2 20260408

```

使用 perf record 和 perf report 分析热点函数:

```

1 | perf record -F 99 -g -o perf_serial.data -- ./serial_O2 20260408
2 | perf report -i perf_serial.data

```

pthread 线程池版本和 OpenMP 版本也采用同样方式生成热点函数报告。

(二) 整体性能统计

三种版本的 perf stat 统计结果如下图所示。

```

Performance counter stats for './serial_O2 20260408':

      100,766.78 msec task-clock:u          #    1.000 CPUs utilized
    259,320,178,048 cycles:u                #    2.573 GHz
    120,544,345,574 instructions:u          #    0.46 insn per cycle
     41,597,188,439 cache-references:u      # 412.807 M/sec
     2,964,916,845 cache-misses:u          #    7.128 % of all cache refs
<not supported> branches:u
     20,190,359 branch-misses:u
           0 context-switches:u           #    0.000 K/sec
           0 cpu-migrations:u              #    0.000 K/sec
       27,473 page-faults:u                #    0.273 K/sec

   100.781314304 seconds time elapsed

   100.379704000 seconds user
     0.075740000 seconds sys

```

图 5: 串行版本 perf stat 统计结果

```

Performance counter stats for './openmp_O2 20260408' (5 runs):

      91,500.14 msec task-clock:u          #    1.070 CPUs utilized    ( +- 21.90% )
    232,953,211,149 cycles:u                #    2.546 GHz              ( +- 22.04% )
    142,534,249,610 instructions:u          #    0.61 insn per cycle    ( +- 0.11% )
     45,268,799,061 cache-references:u      # 494.740 M/sec              ( +- 0.06% )
     2,494,885,769 cache-misses:u          #    5.511 % of all cache refs ( +- 8.46% )
<not supported> branches:u
     19,165,356 branch-misses:u              ( +- 0.12% )
           0 context-switches:u           #    0.000 K/sec
           0 cpu-migrations:u              #    0.000 K/sec
       20,194 page-faults:u                #    0.221 K/sec            ( +- 12.59% )

   85.48 +- 19.73 seconds time elapsed ( +- 23.08% )

```

图 6: OpenMP 版本 perf stat 统计结果

```

Performance counter stats for './pthread_pool_02 20260408':

      47,593.39 msec task-clock:u          #    1.007 CPUs utilized
    119,093,805,136 cycles:u              #    2.502 GHz
    120,552,571,166 instructions:u       #    1.01 insn per cycle
    41,604,383,535 cache-references:u     #   874.163 M/sec
    1,947,331,794 cache-misses:u         #    4.681 % of all cache refs
<not supported> branches:u
    20,825,053 branch-misses:u
           0 context-switches:u          #    0.000 K/sec
           0 cpu-migrations:u            #    0.000 K/sec
       8,031 page-faults:u               #    0.169 K/sec

47.255404957 seconds time elapsed

46.186102000 seconds user
 2.245250000 seconds sys

```

图 7: pthread 线程池版本 perf stat 统计结果

根据 perf 输出整理结果如下表:

表 6: 不同版本 perf stat 性能统计对比

版本	time elapsed	task-clock	CPUs utilized	IPC	Cache miss rate	page-faults
串行版本	100.781s	100766.78ms	1.000	0.46	7.128%	27473
OpenMP 版本	85.48s	91500.14ms	1.070	0.61	5.511%	20194
pthread 线程池版本	47.255s	47593.39ms	1.007	1.01	4.681%	8031

从表中可以看出, 串行版本运行时间约为 100.781s, OpenMP 版本运行时间约为 85.48s, pthread 线程池版本运行时间约为 47.255s。在本次 perf 测试中, pthread 线程池版本的运行时间最低, 说明线程池版本在该输入下取得了较好的优化效果。

从 CPU 利用率来看, OpenMP 版本虽然设置了 4 个线程, 但平均 CPU 利用率只有 1.070, pthread 线程池版本也只有 1.007。这说明程序并没有长时间充分利用 4 个核心。其主要原因是本实验采用的是块间并行、块内串行策略。只有当某一轮 GKH 迭代中存在多个非平凡活动块时, 多个线程才有足够任务可以并行执行; 如果某一轮只有一个主要活动块, 则只有一个线程能够执行核心计算, 其余线程只能等待或没有任务可做。

从 IPC 来看, 串行版本 IPC 为 0.46, OpenMP 版本提高到 0.61, 而 pthread 线程池版本提高到 1.01。pthread 线程池版本的 IPC 更高, 说明其在本次测试中指令执行效率更好。与此同时, Cache miss rate 从串行版本的 7.128% 降低到 OpenMP 版本的 5.511%, 再降低到 pthread 线程池版本的 4.681%。这表明线程池版本不仅减少了总运行时间, 也在一定程度上改善了 Cache 访问表现。

(三) 热点函数分析

为了进一步定位程序主要耗时位置, 使用 perf report 查看热点函数。三种版本的热点函数结果如下图所示。

Children	Self	Command	Shared Object	Symbol
+ 100.00%	0.00%	serial_02	serial_02	[.] start
+ 100.00%	0.00%	serial_02	libc.so.6	[.] __libc_start_main
+ 100.00%	0.00%	serial_02	serial_02	[.] main
+ 99.98%	0.02%	serial_02	serial_02	[.] run_case
+ 62.59%	62.59%	serial_02	serial_02	[.] (anonymous namespace)::apply_right_cols
+ 13.35%	13.35%	serial_02	serial_02	[.] Matrix::operator*
+ 10.59%	10.57%	serial_02	serial_02	[.] to_bidiagonal
+ 9.59%	9.59%	serial_02	serial_02	[.] (anonymous namespace)::cleanup_bidiagonal
+ 6.45%	0.00%	serial_02	serial_02	[.] orth_error
+ 3.46%	3.46%	serial_02	serial_02	[.] (anonymous namespace)::apply_left_rows
+ 0.27%	0.23%	serial_02	serial_02	[.] gkh_svd_from_bidiagonal
+ 0.07%	0.07%	serial_02	libm.so.6	[.] hypotf32x
+ 0.05%	0.05%	serial_02	serial_02	[.] fro_norm
+ 0.02%	0.02%	serial_02	libc.so.6	[.] 0x000000000008fa30
+ 0.02%	0.00%	serial_02	libc.so.6	[.] cfree
+ 0.00%	0.00%	serial_02	libc.so.6	[.] 0x0000000000000000

图 8: 串行版本 perf report 热点函数

Children	Self	Command	Shared Object	Symbol
+ 95.67%	0.00%	openmp_02	openmp_02	[.] start
+ 95.67%	0.00%	openmp_02	libc.so.6	[.] __libc_start_main
+ 95.67%	0.00%	openmp_02	libc.so.6	[.] 0x000000000001b000
+ 95.67%	0.01%	openmp_02	openmp_02	[.] main
+ 95.67%	0.01%	openmp_02	openmp_02	[.] run_case
+ 87.07%	0.31%	openmp_02	openmp_02	[.] gkh_svd_from_bidiagonal
+ 83.88%	83.88%	openmp_02	openmp_02	[.] (anonymous namespace)::apply_right_cols
+ 4.35%	4.35%	openmp_02	openmp_02	[.] to_bidiagonal
+ 4.33%	0.00%	openmp_02	libc.so.6	[.] 0x00000000000ad9d5c
+ 4.33%	0.00%	openmp_02	libc.so.6	[.] 0x00000000000a72518
+ 4.16%	4.16%	openmp_02	openmp_02	[.] Matrix::operator*
+ 4.12%	4.12%	openmp_02	openmp_02	[.] (anonymous namespace)::cleanup_bidiagonal
+ 3.22%	0.00%	openmp_02	libgomp.so.1.0.0	[.] 0x00000000000b97e0
+ 1.94%	0.00%	openmp_02	openmp_02	[.] orth_error
+ 1.54%	1.54%	openmp_02	openmp_02	[.] (anonymous namespace)::apply_left_rows
+ 0.95%	0.00%	openmp_02	libgomp.so.1.0.0	[.] GOMP_parallel
+ 0.80%	0.00%	openmp_02	libgomp.so.1.0.0	[.] 0x00000000000b97c4
+ 0.43%	0.43%	openmp_02	libgomp.so.1.0.0	[.] 0x000000000001c210
+ 0.43%	0.00%	openmp_02	libgomp.so.1.0.0	[.] 0x00000000000bfc210
+ 0.38%	0.38%	openmp_02	libgomp.so.1.0.0	[.] 0x000000000001c4ec
+ 0.38%	0.00%	openmp_02	libgomp.so.1.0.0	[.] 0x00000000000bfc4ec
+ 0.36%	0.36%	openmp_02	libgomp.so.1.0.0	[.] 0x000000000001c21c
+ 0.36%	0.00%	openmp_02	libgomp.so.1.0.0	[.] 0x00000000000bfc21c
+ 0.31%	0.00%	openmp_02	libgomp.so.1.0.0	[.] 0x00000000000b97a8
+ 0.22%	0.00%	openmp_02	libgomp.so.1.0.0	[.] 0x00000000000b9a94
+ 0.15%	0.15%	openmp_02	libgomp.so.1.0.0	[.] 0x000000000001c4e0
+ 0.15%	0.00%	openmp_02	libgomp.so.1.0.0	[.] 0x00000000000bfc4e0
+ 0.10%	0.00%	openmp_02	openmp_02	[.] (anonymous namespace)::one_block_step
+ 0.07%	0.07%	openmp_02	openmp_02	[.] transpose
+ 0.06%	0.06%	openmp_02	libm.so.6	[.] hypotf32x
+ 0.04%	0.00%	openmp_02	openmp_02	[.] gkh_svd_from_bidiagonal
+ 0.03%	0.03%	openmp_02	openmp_02	[.] hypotf32x
+ 0.02%	0.02%	openmp_02	libm.so.6	[.] __hypot_finite
+ 0.01%	0.01%	openmp_02	libgomp.so.1.0.0	[.] 0x000000000001c238
+ 0.01%	0.00%	openmp_02	libgomp.so.1.0.0	[.] 0x00000000000bfc238
+ 0.01%	0.01%	openmp_02	libc.so.6	[.] 0x0000000000000188
+ 0.01%	0.00%	openmp_02	libc.so.6	[.] 0x00000000000b8c188
+ 0.01%	0.01%	openmp_02	openmp_02	[.] std::vector<double, std::allocator<double> >::vector
+ 0.01%	0.01%	openmp_02	openmp_02	[.] operator new<plt>
+ 0.01%	0.01%	openmp_02	libc.so.6	[.] malloc
+ 0.01%	0.00%	openmp_02	openmp_02	[.] std::vector<(anonymous namespace)::Block, std::allocator<(anonymous namespace)::Block> >::M_realloc_insert<(anon
+ 0.01%	0.00%	openmp_02	libstdc++.so.6.0.28	[.] operator new
+ 0.01%	0.01%	openmp_02	libgomp.so.1.0.0	[.] syscall@plt

图 9: OpenMP 版本 perf report 热点函数

Children	Self	Command	Shared Object	Symbol
+ 63.69%	0.00%	pthread_pool_02	libc.so.6	[.] 0x0000000000082c9d5c
+ 63.69%	0.00%	pthread_pool_02	libc.so.6	[.] 0x000000000008262518
+ 58.42%	58.42%	pthread_pool_02	pthread_pool_02	[.] (anonymous namespace)::apply_right_cols
+ 36.31%	0.00%	pthread_pool_02	pthread_pool_02	[.] start
+ 36.31%	0.00%	pthread_pool_02	libc.so.6	[.] __libc_start_main
+ 36.31%	0.00%	pthread_pool_02	libc.so.6	[.] 0x00000000000820b000
+ 36.31%	0.00%	pthread_pool_02	pthread_pool_02	[.] main
+ 36.30%	0.02%	pthread_pool_02	pthread_pool_02	[.] run_case
+ 18.24%	18.24%	pthread_pool_02	pthread_pool_02	[.] (anonymous namespace)::cleanup_bidiagonal
+ 9.93%	9.93%	pthread_pool_02	pthread_pool_02	[.] Matrix::operator*
+ 7.82%	7.81%	pthread_pool_02	pthread_pool_02	[.] to_bidiagonal
+ 5.04%	5.04%	pthread_pool_02	pthread_pool_02	[.] (anonymous namespace)::apply_left_rows
+ 4.63%	0.00%	pthread_pool_02	pthread_pool_02	[.] orth_error
+ 0.24%	0.19%	pthread_pool_02	pthread_pool_02	[.] gkh_svd_from_bidiagonal
+ 0.11%	0.02%	pthread_pool_02	pthread_pool_02	[.] (anonymous namespace)::Pool::worker_entry
+ 0.10%	0.08%	pthread_pool_02	libm.so.6	[.] hypotf32x
+ 0.05%	0.05%	pthread_pool_02	pthread_pool_02	[.] hypotf32x
+ 0.04%	0.00%	pthread_pool_02	libc.so.6	[.] pthread_cond_wait
+ 0.04%	0.04%	pthread_pool_02	libm.so.6	[.] __hypot_finite
+ 0.02%	0.02%	pthread_pool_02	libc.so.6	[.] 0x0000000000007ed4c
+ 0.02%	0.00%	pthread_pool_02	libc.so.6	[.] 0x00000000000825ed4c
+ 0.02%	0.02%	pthread_pool_02	libc.so.6	[.] 0x00000000000086c24
+ 0.02%	0.00%	pthread_pool_02	libc.so.6	[.] 0x000000000008266c24
+ 0.02%	0.02%	pthread_pool_02	libc.so.6	[.] malloc
+ 0.02%	0.02%	pthread_pool_02	libc.so.6	[.] 0x0000000000007e948

图 10: pthread 线程池版本 perf report 热点函数

根据 perf report 中的 Self 开销, 可以整理出主要热点如下:

表 7: 不同版本热点函数占比

版本	apply_right_cols	cleanup_bidiagonal	Matrix::operator*	apply_left_rows	并行运行时/同步函数
串行版本	62.59%	9.59%	13.35%	3.46%	-
OpenMP 版本	83.88%	4.12%	4.16%	1.54%	GOMP_parallel: 0.95%
pthread 线程池版本	58.42%	18.24%	9.93%	5.04%	pthread_cond_wait: 0.04%

从热点函数可以看出, 三个版本的主要开销都集中在矩阵数值计算相关函数中, 尤其是 `apply_right_cols()`。串行版本中 `apply_right_cols()` 的 Self 占比为 62.59%, OpenMP 版本中达到 83.88%, pthread 线程池版本中为 58.42%。这说明程序瓶颈主要来自矩阵列更新等数值计算过程, 而不是线程运行时本身。

OpenMP 版本中 `GOMP_parallel` 的占比约为 0.95%, 说明 OpenMP 运行时确实存在一定调度开销, 但它并不是最主要的性能瓶颈。OpenMP 版本的主要耗时仍然集中在矩阵计算函数中。结合 CPU 利用率只有 1.070 可以看出, OpenMP 版本虽然使用了多线程框架, 但在当前算法结构下, 并行任务数量不足, 导致平均并行度较低。

pthread 线程池版本中, `pthread_cond_wait` 的占比只有 0.04%, 说明在 perf 采样结果中, 条件变量等待并不是主要热点。线程池版本的主要开销仍然来自 `apply_right_cols()`、`cleanup_bidiagonal()` 和 `Matrix::operator*` 等数值计算函数。相比 OpenMP 版本, pthread 线程池版本避免了频繁进入 OpenMP 并行区域的开销, 并通过动态任务领取改善了负载分配, 因此在本次测试中运行时间更低。

(四) 性能结果分析

综合 perf stat 和 perf report 的结果可以发现, 三种版本的性能差异并不完全由线程数量决定, 而是由算法可并行任务数量、线程调度开销和底层矩阵计算效率共同决定。

串行版本没有线程调度开销, 但只能使用单个线程执行所有 GKH 迭代, 因此总运行时间最长。其 IPC 只有 0.46, Cache miss rate 为 7.128%, 说明串行版本在矩阵访问和数据依赖方面存在一定瓶颈。

OpenMP 版本相较串行版本有所加速, 运行时间从 100.781s 降低到 85.48s。其 IPC 提升到 0.61, Cache miss rate 降低到 5.511%。但是, OpenMP 版本的 CPU 利用率只有 1.070, 远低于理想的 4 核并行。这说明当前块间并行策略受限于活动块数量, 并不是每一轮都有足够多的 block 可以分配给多个线程。

pthread 线程池版本运行时间为 47.255s, 在本次测试中明显快于串行版本和 OpenMP 版本。其 IPC 达到 1.01, Cache miss rate 降低到 4.681%。这说明线程池版本在该输入下不仅减少了线程调度开销, 也获得了更好的指令执行效率和缓存表现。线程池版本通过预先创建工作线程、动态领取活动块任务, 避免了非线程池版本中反复创建和销毁线程的问题, 也比 OpenMP 版本更容易针对当前任务结构进行调度控制。

不过, pthread 线程池版本的 CPU 利用率仍然只有 1.007, 说明它并没有真正实现长时间多核满负载运行。这也说明本次加速可能并没真正并行化。原因可能在于当前实现没有对单个活动块内部进一步并行化。当某一轮只有一个主要活动块时, 其他线程无法参与该 block 内部计算, 因此并行度会受到限制。

(五) 小结

perf 分析说明了我们程序的主要瓶颈并不只是线程创建或 OpenMP 调度开销，而是矩阵数值计算本身以及迭代中的块内串行依赖。OpenMP 版本可以通过简单的 `parallel for` 实现块间并行，并在本次测试中相较串行版本取得一定加速；pthread 线程池版本进一步通过线程复用和动态任务领取降低调度开销，在本次 perf 测试中取得了更好的运行时间。但是，从 CPU 利用率可以看出，三种版本都没有充分利用多个核心。后续如果要进一步优化，可能需要突破单纯块间并行的限制，同时也可以继续优化 Cache 局部性，以进一步降低额外开销。

七、 大模型对话记录

Q: 我的电脑使用不了 vtune 进行分析，是否可以使用 perf 进行分析呢？

A: 可以，手册中建议使用 VTune 对多线程程序进行分析，但由于本实验环境无法正常安装或运行 VTune，因此本文采用 Linux perf 工具作为替代。perf 可以采集 task-clock、context-switches、cycles、instructions、IPC 等性能指标，用于分析程序的 CPU 使用情况、线程调度开销和指令执行效率。需要说明的是，perf 与 VTune 的侧重点不同。VTune 更适合观察线程时间线、等待时间和热点函数分布；perf 更偏向命令行性能计数器统计。虽然 perf 不能完全替代 VTune 的图形化线程分析功能，但对于本文比较串行版本、pthread 线程池版本和 OpenMP 版本的整体性能开销已经足够。

Q: 我在跑实验的过程中有时候数据很高是为什么，服务器的原因吗

A: 是的，很可能和服务器环境有关，但不一定只有服务器原因。你这个 SVD 实验运行时间偶尔突然变高，通常由下面几类因素共同造成。1. 服务器共享负载导致波动,2. 多线程调度本身会带来不稳定,3. 不同 seed 本来就会导致迭代次数不同。

八、 代码链接

点击访问 [GitHub 主页](#)